

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

PRACTICAL - 4

Aim :- To implement a digital signature algorithm using RSA and verify the integrity and authenticity of digitally signed messages using CLI.

Code :-

```
import hashlib
import math
# Function to check whether a number is prime
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False
    return True

# Extended Euclidean Algorithm
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    gcd, x1, y1 = extended_gcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1
    return gcd, x, y

# Calculate modular inverse
def modular_inverse(e, phi):
    gcd, x, y = extended_gcd(e, phi)
    if gcd != 1:
        return None
    return x % phi

# Generate SHA-256 hash
def generate_hash(message):
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    return hash_value

# Convert hexadecimal hash into integer
def hash_to_integer(hash_value):
    return int(hash_value, 16)

# Generate RSA keys
def generate_rsa_keys(p, q, e):
    n = p * q
    phi = (p - 1) * (q - 1)
    if math.gcd(e, phi) != 1:
        return None
    d = modular_inverse(e, phi)
    return n, phi, d

# Create digital signature
```

T101

RAJDEEP M PARAB

TY.BSc.CS

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
def create_signature(message, d, n):
    hash_value = generate_hash(message)
    hash_integer = hash_to_integer(hash_value)

    # RSA signature
    signature = pow(hash_integer, d, n)
    return hash_value, signature

# Verify digital signature
def verify_signature(message, signature, e, n):
    hash_value = generate_hash(message)
    hash_integer = hash_to_integer(hash_value)

    # Recover hash from signature
    recovered_hash = pow(signature, e, n)
    # Since small educational RSA keys may not contain the
    # complete SHA-256 integer, compare modulo n.
    if recovered_hash == hash_integer % n:
        return True, hash_value, recovered_hash
    else:
        return False, hash_value, recovered_hash

# ----- MAIN PROGRAM -----
print("=" * 60)
print("    RSA DIGITAL SIGNATURE IMPLEMENTATION")
print("=" * 60)
# Input prime numbers
while True:
    p = int(input("\nEnter first prime number (p): "))
    if is_prime(p):
        break
    else:
        print("Error: p must be a prime number.")
while True:
    q = int(input("Enter second prime number (q): "))
    if is_prime(q) and q != p:
        break
    else:
        print("Error: q must be a different prime number.")

# Calculate n and phi
n = p * q
phi = (p - 1) * (q - 1)
print("\nCalculated RSA values:")
print("n =", n)
print("phi(n) =", phi)

# Select public exponent
while True:
    e = int(input("\nEnter public exponent (e): "))
    if 1 < e < phi and math.gcd(e, phi) == 1:
        break
    else:
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
print("Invalid e. It must satisfy gcd(e, phi) = 1.")

# Calculate private key
d = modular_inverse(e, phi)
print("\nRSA Keys:")
print("Public Key =", (e, n))
print("Private Key =", (d, n))

# Input original message
message = input("\nEnter message to digitally sign: ")

# Generate digital signature
original_hash, signature = create_signature(message, d, n)
print("\n" + "=" * 60)
print("DIGITAL SIGNATURE GENERATION")
print("=" * 60)
print("Original Message :", message)
print("SHA-256 Hash      :", original_hash)
print("Digital Signature:", signature)

# Verification message
verification_message = input(
    "\nEnter message for signature verification: "
)
valid, received_hash, recovered_hash = verify_signature(
    verification_message,
    signature,
    e,
    n
)

print("\n" + "=" * 60)
print("DIGITAL SIGNATURE VERIFICATION")
print("=" * 60)
print("Received Message :", verification_message)
print("SHA-256 Hash      :", received_hash)
print("Recovered Hash     :", recovered_hash)

if valid:
    print("\nResult: DIGITAL SIGNATURE VERIFIED")
    print("The message is authentic and its integrity is maintained.")
else:
    print("\nResult: DIGITAL SIGNATURE VERIFICATION FAILED")
    print("The message may have been modified or the signature is invalid.")

print("\n" + "=" * 60)
print("END OF PRACTICAL")
print("=" * 60)
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

Output :-

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=====
RESTART: C:/Users/mvluc/Documents/T101 Cybersecurity Practical - 4/Practical 4A.py
=====
RSA DIGITAL SIGNATURE IMPLEMENTATION
=====

Enter first prime number (p): 61
Enter second prime number (q): 53

Calculated RSA values:
n = 3233
phi(n) = 3120

Enter public exponent (e): 17

RSA Keys:
Public Key = (17, 3233)
Private Key = (2753, 3233)

Enter message to digitally sign: T101 Rajdeep M Parab

=====
DIGITAL SIGNATURE GENERATION
=====
Original Message : T101 Rajdeep M Parab
SHA-256 Hash      : 4c1035a7e8c64bb32be3ffe811a2120d3d0b7f68ed85cf881dcb250b0882126a
Digital Signature: 210

Enter message for signature verification: T101 Rajdeep M Parab

=====
DIGITAL SIGNATURE VERIFICATION
=====
Received Message : T101 Rajdeep M Parab
SHA-256 Hash      : 4c1035a7e8c64bb32be3ffe811a2120d3d0b7f68ed85cf881dcb250b0882126a
Recovered Hash     : 2382

Result: DIGITAL SIGNATURE VERIFIED
The message is authentic and its integrity is maintained.

=====
END OF PRACTICAL
=====
>>>

Ln: 48 Col: 0
28°C Partly sunny 08:34 10-08-2026

IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
>>>
=====
RESTART: C:/Users/mvluc/Documents/T101 Cybersecurity Practical - 4/Practical 4B.py
=====
>>>
=====
RESTART: C:/Users/mvluc/Documents/T101 Cybersecurity Practical - 4/Practical 4A.py
=====
=====
RSA DIGITAL SIGNATURE IMPLEMENTATION
=====

Enter first prime number (p): 61
Enter second prime number (q): 53

Calculated RSA values:
n = 3233
phi(n) = 3120

Enter public exponent (e): 17

RSA Keys:
Public Key = (17, 3233)
Private Key = (2753, 3233)

Enter message to digitally sign: T101 Rajdeep M Parab

=====
DIGITAL SIGNATURE GENERATION
=====
Original Message : T101 Rajdeep M Parab
SHA-256 Hash      : 4c1035a7e8c64bb32be3ffe811a2120d3d0b7f68ed85cf881dcb250b0882126a
Digital Signature: 210

Enter message for signature verification: T101 Rajdeep parab

=====
DIGITAL SIGNATURE VERIFICATION
=====
Received Message : T101 Rajdeep parab
SHA-256 Hash      : b36fa2699163bc596edd03818a2d5b969053a09de3df8e316bc0b177769044da
Recovered Hash     : 2382

Result: DIGITAL SIGNATURE VERIFICATION FAILED
The message may have been modified or the signature is invalid.

=====
END OF PRACTICAL
=====
>>>

Ln: 91 Col: 0
28°C Partly sunny 08:35 10-08-2026
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

Aim :- To implement a digital signature algorithm using RSA and verify the integrity and authenticity of digitally signed messages using GUI.

Code :-

```
import tkinter as tk
from tkinter import messagebox
import hashlib
import math

# ----- RSA FUNCTIONS -----
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False
    return True
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    gcd, x1, y1 = extended_gcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1
    return gcd, x, y
def modular_inverse(e, phi):
    gcd, x, y = extended_gcd(e, phi)
    if gcd != 1:
        return None
    return x % phi
def generate_hash(message):
    return hashlib.sha256(message.encode()).hexdigest()

# ----- KEY GENERATION -----
def generate_keys():
    try:
        p = int(p_entry.get())
        q = int(q_entry.get())
        e = int(e_entry.get())
        if not is_prime(p):
            messagebox.showerror(
                "Invalid Input",
                "p must be a prime number."
            )
        return
        if not is_prime(q):
            messagebox.showerror(
                "Invalid Input",
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
"q must be a prime number."
)
return
if p == q:
    messagebox.showerror(
        "Invalid Input",
        "p and q must be different prime numbers."
    )
    return
n = p * q
phi = (p - 1) * (q - 1)
if e <= 1 or e >= phi:
    messagebox.showerror(
        "Invalid Input",
        "e must satisfy 1 < e < phi(n)."
    )
    return
if math.gcd(e, phi) != 1:
    messagebox.showerror(
        "Invalid Input",
        "e must be relatively prime to phi(n)."
    )
    return
d = modular_inverse(e, phi)
if d is None:
    messagebox.showerror(
        "Error",
        "Could not calculate private key."
    )
    return

# Store values globally
global rsa_n, rsa_phi, rsa_d, rsa_e
rsa_n = n
rsa_phi = phi
rsa_d = d
rsa_e = e
key_output.delete("1.0", tk.END)
key_output.insert(
    tk.END,
    "RSA KEYS GENERATED SUCCESSFULLY\n\n"
)
key_output.insert(
    tk.END,
    f"p = {p}\n"
    f"q = {q}\n"
    f"n = p × q = {n}\n"
    f"phi(n) = {phi}\n\n"
    f"Public Key = ({e}, {n})\n"
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
f"Private Key = ({d}, {n})\n"
)
messagebox.showinfo(
    "Success",
    "RSA keys generated successfully!"
)
except ValueError:
    messagebox.showerror(
        "Invalid Input",
        "Please enter valid integer values for p, q and e."
    )

# ----- DIGITAL SIGNATURE -----
def generate_signature():
    try:
        if "rsa_n" not in globals():
            messagebox.showwarning(
                "Warning",
                "First generate the RSA keys."
            )
            return
        message = message_entry.get("1.0", tk.END).strip()
        if message == "":
            messagebox.showwarning(
                "Warning",
                "Please enter a message."
            )
            return
        # SHA-256 hash
        hash_value = generate_hash(message)
        # Convert hash to integer
        hash_integer = int(hash_value, 16)
        # RSA signature
        signature = pow(
            hash_integer,
            rsa_d,
            rsa_n
        )
        # Store signature globally
        global digital_signature
        digital_signature = signature
        signature_output.delete("1.0", tk.END)
        signature_output.insert(
            tk.END,
            "DIGITAL SIGNATURE GENERATED\n\n"
        )
        signature_output.insert(
            tk.END,
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
f"Original Message:\n{message}\n\n"
f"SHA-256 Hash:\n{hash_value}\n\n"
f"Digital Signature:\n{signature}\n"
)
messagebox.showinfo(
    "Success",
    "Digital signature generated successfully!"
)
except Exception as error:
    messagebox.showerror(
        "Error",
        str(error)
    )

# ----- SIGNATURE VERIFICATION -----
def verify_signature():
    try:
        if "rsa_n" not in globals():
            messagebox.showwarning(
                "Warning",
                "First generate the RSA keys."
            )
            return
        if "digital_signature" not in globals():
            messagebox.showwarning(
                "Warning",
                "First generate the digital signature."
            )
            return
        verification_message = verify_entry.get(
            "1.0",
            tk.END
        ).strip()
        if verification_message == "":
            messagebox.showwarning(
                "Warning",
                "Please enter a message for verification."
            )
            return

        # Generate hash of received message
        received_hash = generate_hash(
            verification_message
        )
        received_hash_integer = int(
            received_hash,
            16
        )
```


SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
# Recover hash using public key
recovered_hash = pow(
    digital_signature,
    rsa_e,
    rsa_n
)

# Compare modulo n
calculated_hash_mod_n = (
    received_hash_integer % rsa_n
)
verification_output.delete(
    "1.0",
    tk.END
)
verification_output.insert(
    tk.END,
    "DIGITAL SIGNATURE VERIFICATION\n\n"
)
verification_output.insert(
    tk.END,
    f"Received Message:\n"
    f"{verification_message}\n\n"
)
verification_output.insert(
    tk.END,
    f"SHA-256 Hash of Received Message:\n"
    f"{received_hash}\n\n"
)
verification_output.insert(
    tk.END,
    f"Recovered Hash Value:\n"
    f"{recovered_hash}\n\n"
)
verification_output.insert(
    tk.END,
    f"Hash Modulo n:\n"
    f"{calculated_hash_mod_n}\n\n"
)
if recovered_hash == calculated_hash_mod_n:
    verification_output.insert(
        tk.END,
        "STATUS: DIGITAL SIGNATURE VERIFIED\n\n"
        "The message is authentic.\n"
        "The integrity of the message is maintained."
    )
messagebox.showinfo(
    "Verification Successful",
    "DIGITAL SIGNATURE VERIFIED\n\n"
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
        "Message is authentic and has not been modified."
    )
    else:
        verification_output.insert(
            tk.END,
            "STATUS: VERIFICATION FAILED\n\n"
            "The message may have been modified.\n"
            "The digital signature is invalid."
        )
        messagebox.showerror(
            "Verification Failed",
            "DIGITAL SIGNATURE VERIFICATION FAILED\n\n"
            "The message may have been modified."
        )
    except Exception as error:
        messagebox.showerror(
            "Error",
            str(error)
        )

# ----- CLEAR FUNCTION -----
def clear_all():
    p_entry.delete(0, tk.END)
    q_entry.delete(0, tk.END)
    e_entry.delete(0, tk.END)
    message_entry.delete("1.0", tk.END)
    verify_entry.delete("1.0", tk.END)
    key_output.delete("1.0", tk.END)
    signature_output.delete("1.0", tk.END)
    verification_output.delete("1.0", tk.END)
    global rsa_n, rsa_phi, rsa_d, rsa_e
    global digital_signature
    for variable in [
        "rsa_n",
        "rsa_phi",
        "rsa_d",
        "rsa_e",
        "digital_signature"
    ]:
        if variable in globals():
            del globals()[variable]

# ----- GUI WINDOW -----
root = tk.Tk()
root.title(
    "Cybersecurity Practical 4 - RSA Digital Signature"
)
root.geometry("950x850")
root.resizable(True, True)
```

T101

RAJDEEP M PARAB

TY.BSc.CS

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
# ----- TITLE -----
title = tk.Label(
    root,
    text="RSA DIGITAL SIGNATURE",
    font=("Arial", 20, "bold")
)
title.pack(pady=10)
subtitle = tk.Label(
    root,
    text="Cybersecurity Practical 4",
    font=("Arial", 12)
)
subtitle.pack()

# ----- RSA KEY SECTION -----
key_frame = tk.LabelFrame(
    root,
    text="RSA Key Generation",
    font=("Arial", 11, "bold"),
    padx=10,
    pady=10
)
key_frame.pack(
    fill="x",
    padx=20,
    pady=10
)
tk.Label(
    key_frame,
    text="Prime p:"
).grid(row=0, column=0, padx=5, pady=5)
p_entry = tk.Entry(
    key_frame,
    width=15
)
p_entry.grid(row=0, column=1, padx=5)
tk.Label(
    key_frame,
    text="Prime q:"
).grid(row=0, column=2, padx=5, pady=5)
q_entry = tk.Entry(
    key_frame,
    width=15
)
q_entry.grid(row=0, column=3, padx=5)
tk.Label(
    key_frame,
    text="Public e:"
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
).grid(row=0, column=4, padx=5, pady=5)
e_entry = tk.Entry(
    key_frame,
    width=15
)
e_entry.grid(row=0, column=5, padx=5)
generate_key_button = tk.Button(
    key_frame,
    text="Generate RSA Keys",
    command=generate_keys,
    width=20
)
generate_key_button.grid(
    row=0,
    column=6,
    padx=10
)
key_output = tk.Text(
    key_frame,
    height=6,
    width=100
)
key_output.grid(
    row=1,
    column=0,
    columnspan=7,
    padx=5,
    pady=10
)

# ----- MESSAGE SECTION -----
message_frame = tk.LabelFrame(
    root,
    text="Digital Signature Generation",
    font=("Arial", 11, "bold"),
    padx=10,
    pady=10
)
message_frame.pack(
    fill="x",
    padx=20,
    pady=10
)
tk.Label(
    message_frame,
    text="Enter Message:"
).pack(
    anchor="w"
)
```

T101

RAJDEEP M PARAB

TY.BSc.CS

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
message_entry = tk.Text(
    message_frame,
    height=3,
    width=100
)
message_entry.pack(
    pady=5
)
signature_button = tk.Button(
    message_frame,
    text="Generate Digital Signature",
    command=generate_signature,
    width=30
)
signature_button.pack(
    pady=5
)
signature_output = tk.Text(
    message_frame,
    height=9,
    width=100
)
signature_output.pack(
    pady=5
)

# ----- VERIFICATION SECTION -----
verify_frame = tk.LabelFrame(
    root,
    text="Digital Signature Verification",
    font=("Arial", 11, "bold"),
    padx=10,
    pady=10
)
verify_frame.pack(
    fill="x",
    padx=20,
    pady=10
)
tk.Label(
    verify_frame,
    text="Enter Message to Verify:"
).pack(
    anchor="w"
)
verify_entry = tk.Text(
    verify_frame,
    height=3,
    width=100
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
)  
verify_entry.pack(  
    pady=5  
)  
verify_button = tk.Button(  
    verify_frame,  
    text="Verify Digital Signature",  
    command=verify_signature,  
    width=30  
)  
verify_button.pack(  
    pady=5  
)  
verification_output = tk.Text(  
    verify_frame,  
    height=10,  
    width=100  
)  
verification_output.pack(  
    pady=5  
)
```

----- CONTROL BUTTONS -----

```
control_frame = tk.Frame(root)  
control_frame.pack(  
    pady=10  
)  
clear_button = tk.Button(  
    control_frame,  
    text="Clear All",  
    command=clear_all,  
    width=15  
)  
clear_button.pack(  
    side="left",  
    padx=10  
)  
exit_button = tk.Button(  
    control_frame,  
    text="Exit",  
    command=root.destroy,  
    width=15  
)  
exit_button.pack(  
    side="left",  
    padx=10  
)
```

----- START GUI -----

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

root.mainloop()

Output :-

The screenshot displays the 'RSA Digital Signature' application interface. It is divided into three main sections: 'RSA Key Generation', 'Digital Signature Generation', and 'Digital Signature Verification'. In the 'RSA Key Generation' section, the 'Generate RSA Keys' button has been clicked, resulting in a message box stating 'RSA KEYS GENERATED SUCCESSFULLY' with the following parameters: $p = 61$, $q = 53$, $n = p \times q = 3233$, and $\phi(n) = 3120$. The 'Digital Signature Generation' section shows the 'Enter Message' field containing 'T101 Rajdeep M Parab'. The 'Generate Digital Signature' button has been clicked, and the output displays the 'Original Message', the 'SHA-256 Hash', and the 'Digital Signature: 210'. A 'Verification Successful' dialog box is overlaid, stating 'DIGITAL SIGNATURE VERIFIED' and 'Message is authentic and has not been modified.' The 'Digital Signature Verification' section shows the 'Enter Message to Verify' field containing 'T101 Rajdeep M Parab'. The 'Verify Digital Signature' button has been clicked, and the same 'Verification Successful' dialog box is displayed. The application window is titled 'RSA DIGITAL SIGNATURE' and 'Cybersecurity Practical 4'. The system tray at the bottom shows the date and time as 10-08-2026, 08:41.

This screenshot shows the same application interface as the first one, but with a different message and a failed verification result. In the 'Digital Signature Generation' section, the 'Enter Message' field now contains 'T101 Rajdeep M Parab' (with a typo). The 'Generate Digital Signature' button has been clicked, and the output displays the 'Original Message', the 'SHA-256 Hash', and the 'Digital Signature: 210'. A 'Verification Failed' dialog box is overlaid, stating 'DIGITAL SIGNATURE VERIFICATION FAILED' and 'The message may have been modified.' The 'Digital Signature Verification' section shows the 'Enter Message to Verify' field containing 'T101 Rajdeep parab' (with a typo). The 'Verify Digital Signature' button has been clicked, and the same 'Verification Failed' dialog box is displayed. The application window is titled 'RSA DIGITAL SIGNATURE' and 'Cybersecurity Practical 4'. The system tray at the bottom shows the date and time as 10-08-2026, 08:41.

T101
RAJDEEP M PARAB
TY.BSc.CS